Cluster Computing (2026) 29:146
https://doi.org/10.1007/s10586-025-05908-7

Optimizing Federated Learning based on Weighted Hyperparameter Aggregation in Energy-constrained Building Construction

Sa Jim Soe Moe[1] · Anam Nawaz Khan[3] · Anh Tuan Nguyen[1] · Do Hyeun Kim[1,2]

Received: 16 April 2025 / Revised: 30 October 2025 / Accepted: 25 December 2025
© The Author(s) 2026, modified publication 2026

Abstract
Federated learning (FL) is compelling for construction safety because sensitive, site-specific data can remain on devices; however, hyperparameter optimization (HPO) in FL is often impractical on resource-constrained edge nodes due to multi-round tuning and communication overhead. Existing FL-HPO approaches (e.g., weight sharing and server-only selection) either incur iterative client-server loops or underutilize client heterogeneity, thereby raising privacy and governance concerns. We propose an *asynchronous, one-shot* local Bayesian optimization on each client using a small data subset, followed by *loss-weighted aggregation* of client-proposed hyperparameters on the server to produce a single global set for collaborative training. The pipeline is training-method agnostic and integrated into a Digital Twin (DT) platform for device registration, orchestration, and telemetry. Across public datasets (MNIST, EEGEyeState, Weather) and a private worker-safety dataset, our approach improves convergence and accuracy/MAE relative to simple averaging and reduces response time on Raspberry Pi clients, yielding an average improvement of 6.44% in end-to-end performance while remaining deployable on constrained AIoT hardware. In safety prediction, faster convergence and lower error enable earlier and more reliable hazard indicators, with on-device inference supporting near–real-time alerts to supervisors and wearables. We also discuss scalability, privacy of shared hyperparameters, and task complexity as directions for future work in safety-critical FL.

Keywords Artificial intelligence of things · Construction safety · Federated learning · Hyperparameter optimization · Worker safety

1 Introduction

Hyperparameters are critical configurations in machine learning that shape learning dynamics and model performance. Key examples include the number of epochs (full passes over the dataset), batch size (updates per mini-batch), and learning rate (parameter update magnitude) [12, 57]. The goal of hyperparameter optimization (HPO) is to find the hyperparameter set λ that minimizes the validation error E of a learning algorithm A trained on data D_t and evaluated on D_v:

$$\lambda^* = \underset{\lambda \in \lambda^n}{\arg\min}\ ; ED_v\left[AD_t(\lambda)\right]. \tag{1}$$

In real deployments, data are often non-independent and non-identically distributed (non-IID), reflecting heterogeneous and imbalanced sources [49]. Training seeks patterns from such data [41], and HPO refines performance by exploring candidate hyperparameters using methods ranging from grid/random search [8] to more sophisticated approaches.

✉ Do Hyeun Kim
kimdh@jejunu.ac.kr

Sa Jim Soe Moe
sajim22@stu.jejunu.ac.kr

Anam Nawaz Khan
Akhan59@utk.edu

Anh Tuan Nguyen
tuan.na@stu.jejunu.ac.kr

[1] Department of Computer Engineering, Jeju National University, Jeju 63243, Republic of Korea

[2] Bigdata Research Center, Jeju National University, Jeju 63243, Republic of Korea

[3] Department of Industrial and Systems Engineering, University of Tennessee, Knoxville, TN 37996, USA

Published online: 12 February 2026

In construction safety, sites combine dynamic hazards (fall risks, struck-by equipment, heat, and weather exposure) with heterogeneous sensing (wearables, tool telemetry, location beacons, environmental and structural sensors) [21]. Data are privacy-sensitive (worker physiology/location) and highly non-IID across crews, sites, and shifts [34, 40]. Practical systems must deliver low-latency risk scoring for preventive alerts, yet edge devices (e.g., Raspberry Pi class) face tight compute, memory, and power budgets [50]. These constraints directly motivate FL with on-device inference but make hyperparameter selection especially costly: repeated, communication-heavy tuning can delay deployment, and exchanging intermediate model states may raise privacy or governance concerns within contractors and unions. Effective FL-HPO for safety must therefore minimize rounds, protect privacy, and remain robust under heterogeneous devices and data [46]. However, current FL-HPO methods either require iterative, multi-round coordination or centralize key decisions, which are misaligned with edge constraints and can underuse client heterogeneity. This creates a practical gap for safety-critical, resource-limited deployments.

This study focuses on continuous safety-index prediction (site- and worker-level risk scoring) rather than on Personal Protective Equipment (PPE) compliance checks or unsafe behavior detection. The model takes in three input categories: (i) Weather – wind speed, temperature, precipitation; (ii) Building/structural – load, strain, inclination; and (iii) Worker/physiological & location – heart rate and position. The predicted safety index supports threshold-based alerts for wearables and dashboards. Other safety functions (e.g., PPE detection, video analytics) are beyond the scope of this paper.

While HPO is well explored in centralized settings, it is challenging in FL, where data remain distributed across clients [27]. Exhaustive or iterative HPO can be resource-intensive due to repeated training and communication [61]. Simple, parallelizable strategies (e.g., random search) are therefore strong baselines in FL [35], aligning with practical constraints in emerging applications (e.g., transportation). Related work in FL spans algorithmic advances, heterogeneity handling, personalization, fairness, systems, and privacy [62]. However, comparatively fewer studies focus squarely on FL-HPO [25]. Federated Bayesian Optimization (FBO) [13] leverages cross-client information to improve a single participant, rather than yielding global hyperparameters for a collaborative model. Other efforts adjust specific hyperparameters such as the learning rate in SGD-based FL [42]. FedEx draws on NAS weight-sharing to guide hyperparameter selection under successive halving [28], and FLoRa explores single-shot, server-side HPO [65].

Limitations of existing FL-HPO: Current FL-HPO approaches leave three practical gaps for safety-critical, edge deployments. First, many rely on *iterative, multi-round* tuning (e.g., Bayesian optimization loops or successive halving), which increases client–server traffic and latency. Second, *weight-sharing/NAS-style* methods (e.g., FedEx) often assume SGD-centric pipelines and exchange model artifacts that may be undesirable for privacy/governance. Third, *server-only single-shot* schemes (e.g., FLoRa) reduce communication but can neglect client-side heterogeneity during proposal, yielding suboptimal global settings under non-IID data. These gaps motivate a design that (i) performs lightweight client-local search once, (ii) aggregates proposals without exposing model weights, and (iii) remains optimizer-agnostic and heterogeneity-aware. We next categorize prior approaches to situate our method in context. We group prior work into four categories. (A) Centralized HPO (grid, random, BO, evolutionary; multi-fidelity Hyperband/BOHB). Highly effective with full data access, but their iterative evaluations become impractical under FL's communication and privacy constraints. (B) FL with NAS/weight sharing (e.g., FedEx and derivatives). Reduce search cost by sharing weights/architectures, yet typically assume SGD-centric training and require exchanging model artifacts, which elevates privacy/governance risk and device overhead. (C) Adaptive/learning-based HPO for FL (RL, meta-learning, per-client tuning, clustering, encrypted aggregation). These approaches tailor hyperparameters to clients or rounds, but often retain multi-round communication and, in many cases, rely on model access or heavier control logic that strains edge devices. (D) Single-shot or server-focused HPO for FL (e.g., FLoRa). Cuts communication by collapsing to server-side selection; however, it can underuse client-side heterogeneity during proposal and may inherit privacy concerns from shared hyperparameters.

In contrast to these, our approach is training-method agnostic (applicable to SGD and non-SGD) and aims to minimize communication and on-device compute by consolidating client proposals into a *single global hyperparameter set*. We propose *asynchronous, one-shot local Bayesian optimization* on each client using a small local subset, and a *loss-weighted, Top-K aggregation* at the server to produce one global configuration without sharing model weights. The pipeline is operationalized in a *Digital Twin (DT) platform* for device registration, orchestration, and telemetry.

Experiments on Raspberry Pi hardware and multiple datasets, including a worker-safety use case, show consistent gains over simple averaging with low overhead; clients use only a subset of local data during HPO, and the aggregated hyperparameters are then used for federated training.

We study FL hyperparameter optimization for worker-safety index prediction under (i) non-IID data, (ii) strict privacy/governance, and (iii) constrained edge devices. The goal is a *single global hyperparameter set* that improves convergence and accuracy/MAE while keeping client computation and communication low. The main contributions of this paper are:

1. An asynchronous one-shot local BO for FL-HPO on constrained clients, tuning on small local subsets to reduce compute and rounds.
2. Loss-weighted, Top-K hyperparameter aggregation that yields a single global set, improving convergence and accuracy/MAE over simple averaging.
3. DT-orchestrated FL pipeline that registers devices, schedules HPO/training, and monitors telemetry on real Raspberry Pi clients.
4. Empirical study across IID/non-IID partitions and multiple datasets (including a worker-safety use case), showing consistent improvements and near-real-time inference on devices.

The remainder of the paper is organized as follows: Section 2 summarizes related studies. Section 3 details the proposed weighted hyperparameter aggregation mechanism. Sections 4 and 5 present implementation and performance results. Section 6 concludes with limitations and future research directions.

2 Related work

2.1 Digital twin and sensing for construction safety

Construction sites are dynamic, hazard-rich environments, making them a natural target for DT-enabled monitoring. Prior work has linked IoT and DT platforms to track the state and location of workers in real time via RFID/BLE infrastructures [64]. Wearable DT systems using IMUs have captured posture and movement and delivered immediate feedback (e.g., via AR) to reduce unsafe kinematics [4]. DT-driven behavioral simulations have been used to study how workers respond to safety interventions and to inform training design [17]. Closed-loop DT frameworks [47] that integrate machine learning and case-based reasoning have been explored for proactive safety control during live projects. Task-specific DT applications, such as for crane/hoisting operations, have modeled spatiotemporal risk and visualized evolving hazards on site [37]. Together, these initiatives show that DTs have great potential for safety management; however, many systems target narrow use cases, operate at limited scale, or rely on centralized processing that raises privacy and latency concerns in outdoor, multi-worker contexts. Our study complements this line of work by focusing on on-device safety-index prediction under federated training, and by proposing an FL-HPO mechanism that respects privacy, bandwidth, and heterogeneity constraints common to construction deployments.

2.2 Classic hyperparameter optimization techniques in centralized settings

HPO is indispensable for system enhancement when objective functions are non-analytical, non-convex, or non-differentiable. It involves tuning hyperparameters, set prior to training, that significantly impact model performance. In centralized learning, common HPO techniques include grid search [5], random search (RS) [18], Bayesian Optimization (BO) [55], and evolutionary optimization [29]. These methods, while effective, rely on iterative trial-and-error processes to strike a balance between exploration and exploitation, making them computationally expensive. Model-free approaches, such as grid search and RS, explore predefined configurations without relying on parameter models, offering simplicity but often with lower efficiency. In contrast, model-based methods like BO leverage surrogate models to approximate the objective function, resulting in more efficient search processes [48]. The primary distinctions among these approaches lie in the choice of surrogate models and their strategies for selecting subsequent evaluation points [22, 36]. Evolutionary algorithms (EAs), including differential EAs [2, 52], iteratively refine populations to achieve optimal performance.

The high computational costs associated with training deep neural networks on large datasets make full-fidelity BO evaluations impractical. To mitigate this, researchers often employ strategies such as training on smaller data subsets or reducing the number of iterations to balance accuracy and computational efficiency. Hyperband [33], using the Successive Halving Algorithm (SHA) [23], exemplifies a multi-fidelity approach by iteratively refining configurations.

However, SHA's random selection of initial candidates limits optimization precision. Integrating BO with Hyperband, as demonstrated by [16], enhances search efficiency by leveraging insights from prior SHA results. In AutoML, comparative studies have simplified the assessment of HPO methods and improved reproducibility [20]. Tools like HPO-B [9] facilitate the evaluation of transfer-HPO techniques, while HPOBench [15] addresses the need for multi-fidelity HPO benchmarks, offering a robust framework for method comparison. Although FL is becoming important in improving worker safety in outdoor construction situations, most HPO benchmarks center on centralized models, therefore ignoring the challenges of distributed settings.

2.3 Hyperparameter optimization in federated learning

At a high level, prior FL-HPO work falls into four buckets: (A) NAS/weight sharing (e.g., FedEx and derivatives), (B) adaptive/learning-based HPO (RL/meta-learning, per-client tuning, clustering/offloading such as FedCust), (C) single-shot/server-side selection (e.g., FLoRa), and (D) other decentralized or proxy schemes. In FL, the decentralized nature of training amplifies the communication overhead of traditional HPO: methods effective centrally (e.g., BO) become costly due to iterative client–server loops [54]. Several decentralized/proxy approaches (e.g., HyperDistill [32]) aim to adapt hyperparameters rapidly with limited data, but they still incur high communication costs in privacy-sensitive settings. Meta-learning (e.g., MAML [10]) likewise struggles with iteration and privacy in FL. *(A) NAS/weight sharing.* FedEx-style wrappers [26] reduce search cost by sharing weights/architectures, but typically assume SGD-centric pipelines and exchange model artifacts, which can raise privacy/model-access concerns [56, 63]. Offline NAS variants [58, 66] also incur heavy client–server exchange and limit client participation. *(B) Adaptive/learning-based HPO.* Works such as Auto-FedRL [19], FedHPO [30], FedTune [60], and Genetic CFL [3], and clustering/offloading frameworks like FedCust [59] adjust hyperparameters per round/client or group; they can reduce convergence time but often retain multi-round control traffic and may require model/state access. Privacy-preserving variants (e.g., PRIVTUNA [39]) mitigate leakage but add cryptographic overhead. *(C) Single-shot/server-side.* FLoRa [65] aggregates client hyperparameters into a server loss surface and selects a single global setting, sharply reducing rounds. However, it relies on default search spaces and shared HPs that can leak information; it also underutilizes client-side heterogeneity during proposal.

Despite these advances, many approaches remain communication-heavy, assume access to model artifacts, or are brittle on constrained AIoT devices. In contrast, our method is designed for *edge-constrained, privacy-sensitive* deployments:

- One-shot local BO on each client: tune on a small local subset (no multi-round loops) to keep client compute and uplink traffic low.
- Loss-weighted Top-K aggregation of hyperparameters (not weights): consolidate only the best proposals into a single global HP set, preserving client specialization during proposal while avoiding model-artifact exchange.
- DT-assisted orchestration: a lightweight control surface to register devices, schedule HPO/training, and collect telemetry–no additional learning logic on the server.

Compared with NAS/weight sharing (FedEx), we avoid model-artifact exchange and SGD-specific wrappers; compared with adaptive RL/meta approaches, we avoid multi-round control and model access; compared with server-only single-shot (FLoRa), we retain client heterogeneity during proposal yet still commit to one global HP via robust, loss-aware aggregation.

In short, existing FL-HPO methods tend to be communication-heavy, require model artifacts, or underuse client heterogeneity. Our design targets edge constraints by one-shot local BO and loss-weighted Top-K HP aggregation (no weight sharing), coordinated via a lightweight DT layer.

3 Proposed methodology

The conceptual framework for the proposed weighted hyperparameter aggregation mechanism aimed at enhancing model prediction in federated learning is depicted in Figure 1. The process includes two phases: the HPO phase and the FL phase. The HPO phase begins with each client initializing its hyperparameter search space and defining an objective function, typically based on performance metrics like loss. Clients then apply Bayesian optimization to determine optimal local hyperparameters. Upon finding these, clients evaluate their local model performance and transmit both the optimized hyperparameters and corresponding loss values to the federated server.

The server aggregates the hyperparameters and loss values from all clients, assigns weights based on the loss values, and selects the top K clients. We use $K = \lceil 0.3 \cdot N \rceil$ unless noted; Sec. 5 keeps K fixed per experiment. The hyperparameters from these top K clients are aggregated

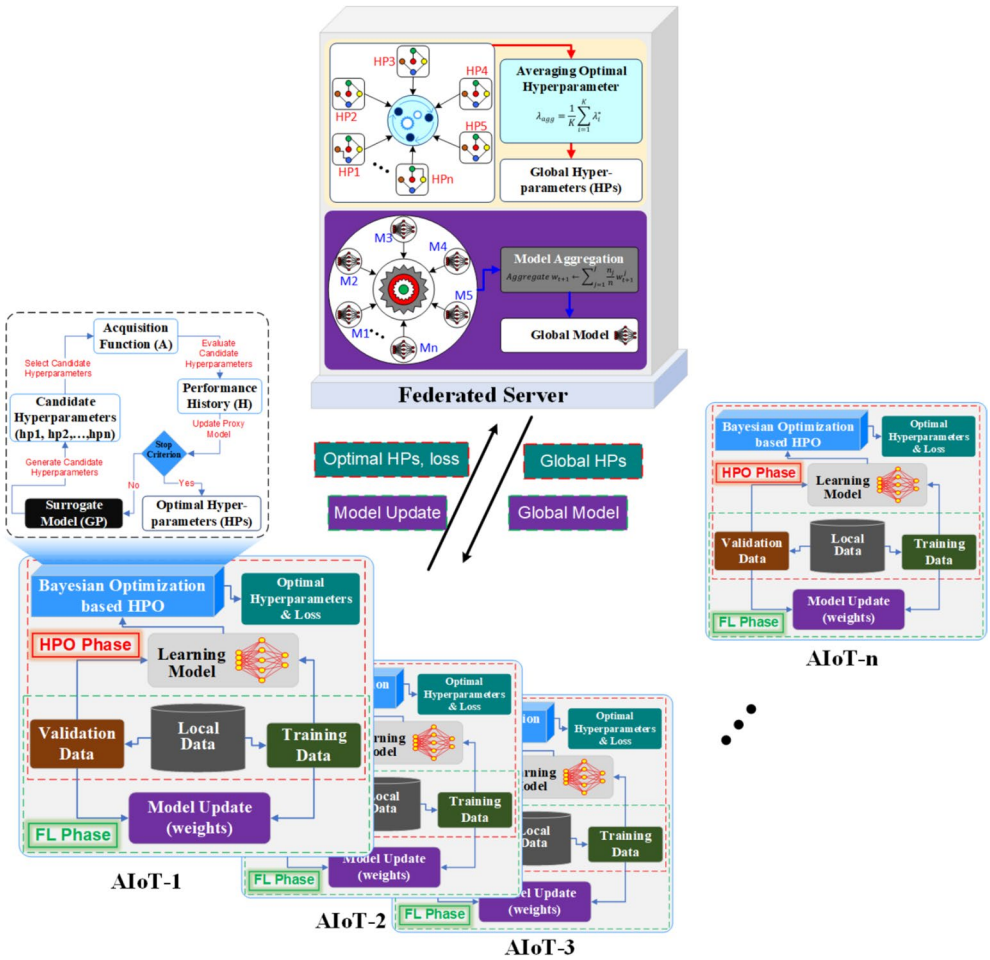


Fig. 1 Conceptual design of weighted hyperparameters aggregation based on enhanced model prediction in federated learning for worker safety

to form a global hyperparameter set, which is distributed to all clients. The FL phase begins with clients using these global hyperparameters to train their local models on their respective data. After training, they send model updates to the server, which aggregates these using federated averaging (FedAVG) to construct a global model. Training iterates with FedAvg; hyperparameters remain fixed after the HPO phase.

Hyperparameters control key aspects of a machine learning model's learning process. These include setting the number of epochs (iterations over the dataset), batch size (data processed per update), learning rate (adjustment speed), and activation function (introducing non-linearity). Choosing these hyperparameters directly impacts model accuracy and efficiency, making HPO essential. In centralized learning, HPO selects the optimal hyperparameters $\lambda_i \in \lambda_i^{(n)}$ to minimize a loss function L. However, in FL, clients with system and statistical heterogeneity collaborate to train a global model, which complicates hyperparameter tuning. Traditional centralized HPO approaches don't work due to the decentralized nature of FL and data privacy concerns. To tackle this, we need a method that actively optimizes hyperparameters in a decentralized, privacy-preserving way to enhance global model performance.

Our proposed method addresses this challenge by incorporating a weighted aggregation approach in the HPO phase. Initially, as stage 1, each client i independently performs HPO using Bayesian optimization with some trials [48, 51] to determine its local best hyperparameters λ_i^* and the corresponding loss value L_i^* (Mean Squared Error, MSE). Next, each client i sends their λ_i^* and L_i^* to the server. The detailed steps of finding the local-best hyperparameters can be found in Algorithm 1.

Input: Local dataset D_i for client i, Initial set of hyperparameters $\{\lambda_i^0, \lambda_i^1, \ldots \lambda_i^n\}$
Output: Best local hyperparameters λ_i^*, Corresponding loss L_i^*
Initialize:
    Set iteration count $t = 0$
    Sample initial hyperparameters $\{\lambda_i^0, \lambda_i^1, \ldots, \lambda_i^n\}$
    Evaluate initial losses $\{L_i^0, L_i^1, \ldots, L_i^n\}$
    Set $D_i^{HPO} = \left\{(\lambda_i^j, L_i^j)\right\}_{j=0}^{n}$
While: stopping criterion not met ***do:***
    Fit GP model to D_i^{HPO}
    Suggest next hyperparameters $\lambda_i^{(t+1)}$ using acquisition function
    Evaluate $L_i^{(t+1)}$ and $\lambda_i^{(t+1)}$ on D_i
    Update $D_i^{HPO} \leftarrow D_i^{HPO} \bigcup \left\{(\lambda_i^{(t+1)}, L_i^{(t+1)})\right\}$
End *While:*
Select Best Hyperparameters:
    $\lambda_i^* = \arg\min_{\lambda} \left\{\lambda_i^j\right\}_{j=0}^{t}$
    $L_i^* = \min \left\{L_i^j\right\}_{j=0}^{t}$
Send Results to Server:
    Transmit λ_i^* and L_i^* to server

Algorithm 1 Hyperparameter optimization on clients with bayesian optimization

Afterward, for stage 2 in the HPO phase, pairs from each client (λ_i^*, L_i^*) are received on the server. The server then calculates a weight w_i from the loss L_i^* (lower loss → higher weight):

$$w_i = \frac{1}{L_i^* + \epsilon}, \tag{2}$$

where $\epsilon > 0$ prevents division by zero.

Upon completion of weight calculation, the server selects the Top-K clients by descending w_i. Let $\mathcal{S}$ denote the indices of the selected clients (or, equivalently, the selected pairs $\{(\lambda_i^*, w_i)\}_{i \in \mathcal{S}}$). Numeric HPs use the weighted mean (rounded if integer); categorical HPs use the weighted mode (ties broken by lower L_i^*, then client ID). These

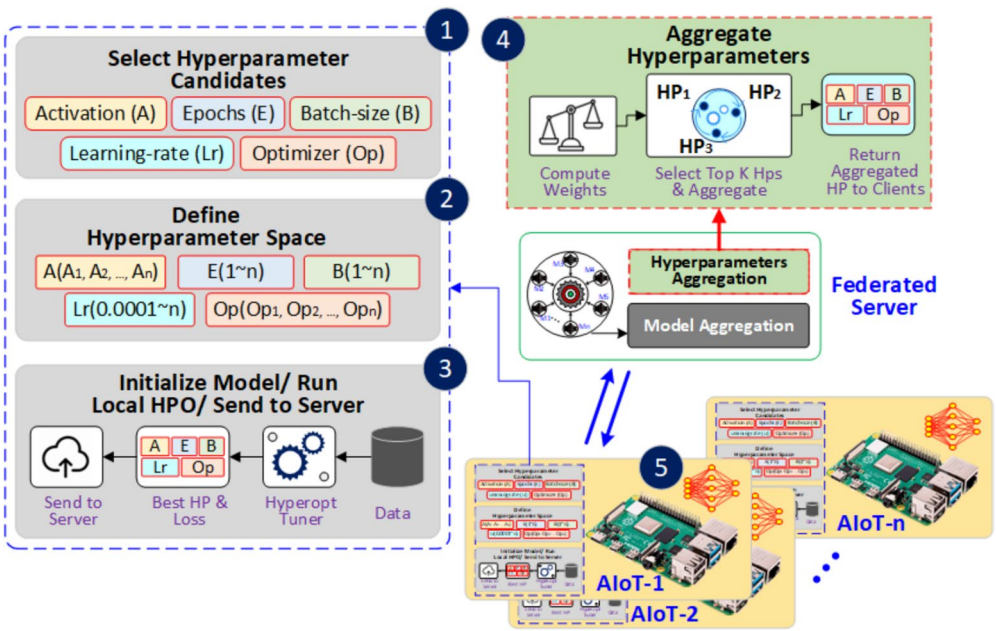


Fig. 2 Procedural diagram of weighted hyperparameter aggregation in FL

hyperparameters are aggregated with normalized weights to produce global hyperparameters λ_{agg}:

$$\tilde{w}_i = \frac{w_i}{\sum_{j\in\mathcal{S}} w_j}, \qquad \lambda_{\text{agg}} = \sum_{i\in\mathcal{S}} \tilde{w}_i\, \lambda_i^*. \tag{3}$$

We set K via a simple rule: default $K = \lceil 0.3\,N \rceil$ (min K=3) kept fixed per experiment. A robust variant (useful under non-IID) keeps all clients with $w_i \geq \text{median}(w) + 0.5 \cdot \text{IQR}(w)$. Finally, the server broadcasts λ_{agg} to all clients for the FL training phase (Algorithm 2).

After local training, clients send their updated model parameters $w_i^{(r+1)}$ to the server. The server consolidates these local model parameters using the FedAVG algorithm [38]:

$$w_{\text{global}}^{(r+1)} = \frac{1}{N}\sum_{i=1}^{N} w_i^{(r+1)} \tag{5}$$

This iterative process continues for a predetermined number of rounds R or until the global model converges to a satisfactory performance level. By utilizing the globally optimized hyperparameters, the training process is expected to be more efficient and result in a more accurate and generalized global model. This two-phase approach ensures that the federated learning process benefits from both local optimizations and global coherence, leading to improved overall model performance. The following Algorithm 3 provides an abstract representation of the detailed procedure.

Input: Client proposals $\{(\lambda_i^*, L_i^*)\}_{i=1}^N$, small constant $\epsilon > 0$, Top-K rule
Output: Aggregated hyperparameters λ_{agg}
Initialize:
    Receive $\{\lambda_i^*, L_i^*\}$ from each client i
Weight Calculation:
    For each client i ***do:***
        $w_i \leftarrow \dfrac{1}{L_i^* + \epsilon}$
    End ***For:***
Select Top-K:
    let $\mathcal{S}$ be indices of the K largest w_i
Normalize weights:
    for $i \in \mathcal{S}$, $\tilde{w}_i \leftarrow \dfrac{w_i}{\sum_{j\in\mathcal{S}} w_j}$
Aggregate numerics:
    for each numeric HP dimension d, $\lambda_{\text{agg}}^{(d)} \leftarrow \sum_{i\in\mathcal{S}} \tilde{w}_i\, \lambda_i^{*(d)}$ (round if integer)
Aggregate categoricals:
    for each categorical HP c, $\lambda_{\text{agg}}^{(c)} \leftarrow \arg\max_v \sum_{i\in\mathcal{S}} \tilde{w}_i\, \mathbf{1}\{\lambda_i^{*(c)} = v\}$
Broadcast:
    send λ_{agg} to all clients for the FL training phase

Algorithm 2 Weighted hyperparameter aggregation on server

In the FL phase, federated learning proceeds using the global hyperparameters obtained from stage 2 of the HPO phase. Each client i adopts these global hyperparameters λ_{agg} and trains its local model on its local dataset $\{D_i\}_{i=1}^N$. The objective of the local training for each client i is:

$$w_i^{(r+1)} = \arg\min_{w_i} L_i(w_i; \lambda_{agg}) \tag{4}$$

where L_i is the local loss function with parameters w_i and global hyperparameters λ_{agg}.

Input: Aggregated hyperparameters λ_{agg}, Local Dataset $\{D_i\}_{i=1}^{N}$, Server rounds R
Output: Converged global model weights w_{global}^{R}
Initialize:
    $\lambda_i \leftarrow \lambda_{agg}, \forall_i \in \{1, \ldots, N\}$
    For each round $r = 0, 1, \ldots, R-1$ ***do***
        Client-side Training:
            For each client $i \in \{1, \ldots, N\}$ ***do***
                Train local model on D_i using λ_i to obtain $w_i^{(r+1)}$
                Send $w_i^{(r+1)}$ to the server
            End *For*
        Server-side Aggregation:
            Aggregate model weights:
                $w_{\text{global}}^{(r+1)} = \frac{1}{N} \sum_{i=1}^{N} w_i^{(r+1)}$
            Send λ_{agg} to all clients
    End *For*
Convergence Check:
    Check ***if*** $\| w_{\text{global}}^{(r+1)} - w_{\text{global}}^{(r)} \| \leq \epsilon$
    if true, set $R = r + 1$
Final Model Distribution:
    Transmit converged model weights w_{global}^{R} to all clients

Algorithm 3 Federated learning process using FedAVG

The following Fig. 2 explains the detailed procedural diagram of weighted hyperparameter aggregation in FL.

1. Select Hyperparameter Candidates: This first step involves identifying different hyperparameters to tune, including activation functions (A), number of epochs (E), batch size (B), learning rate (Lr), and optimizer (Op).
2. Define Hyperparameter Space: Here, a range of values for each hyperparameter is defined. For instance, activation functions could range from A1 to An, epochs might vary logarithmically from 1 to n, batch sizes from 1 to n, learning rates from 0.0001 to n, and optimizers from Op1 to Opn.
3. Initialize Model/Run Local HPO/Send to Server: In this phase, clients initialize their model and perform local HPO. They use a Hyperopt Tuner to determine the best combination of hyperparameters and associated loss, which is then uploaded to the FL server house in the DT platform.
4. Aggregate Hyperparameters: On the server, the hyperparameters' weights are computed, potentially based on their Mean Squared Error (MSE) performance metrics. The Top-*K* hyperparameter sets (HP1, HP2, HP3) are selected and aggregated. The aggregated hyperparameters are sent back to the clients from the server.
5. Collaborative Training: After receiving the global hyperparameters, each node starts the collaborative training, and the following steps fall into the FL configuration.

The following Fig. 3 shows the layer-wise process flow of weighted hyperparameter aggregation in FL integrated with DT. DT technology [6] has rapidly advanced and evolved, particularly in the context of Industry 4.0. A DT is a virtual representation of a physical object or system, enabling real-time monitoring, simulation, and optimization. This technology is pivotal in various sectors, including manufacturing, healthcare, smart cities, and energy systems [53].

The top layer is the Digital Twin (DT) platform, which manages the workflow: it registers physical devices (FL clients), creates their virtual twins, activates them, and establishes two-way communication to form the digital twin network (DTN). During the HPO phase for clients, each defines its hyperparameter search space, generates candidates, and runs a one-time local Bayesian optimization on a small local subset. The best local configuration (HP, loss) is sent back to the server. Only hyperparameters and the associated loss are shared–no model weights. During the HPO phase for the server, the federated server collects all client proposals, calculates loss-based weights, ranks candidates, and selects the Top-*K*. These hyperparameters are then combined (loss-weighted) into a single global hyperparameter

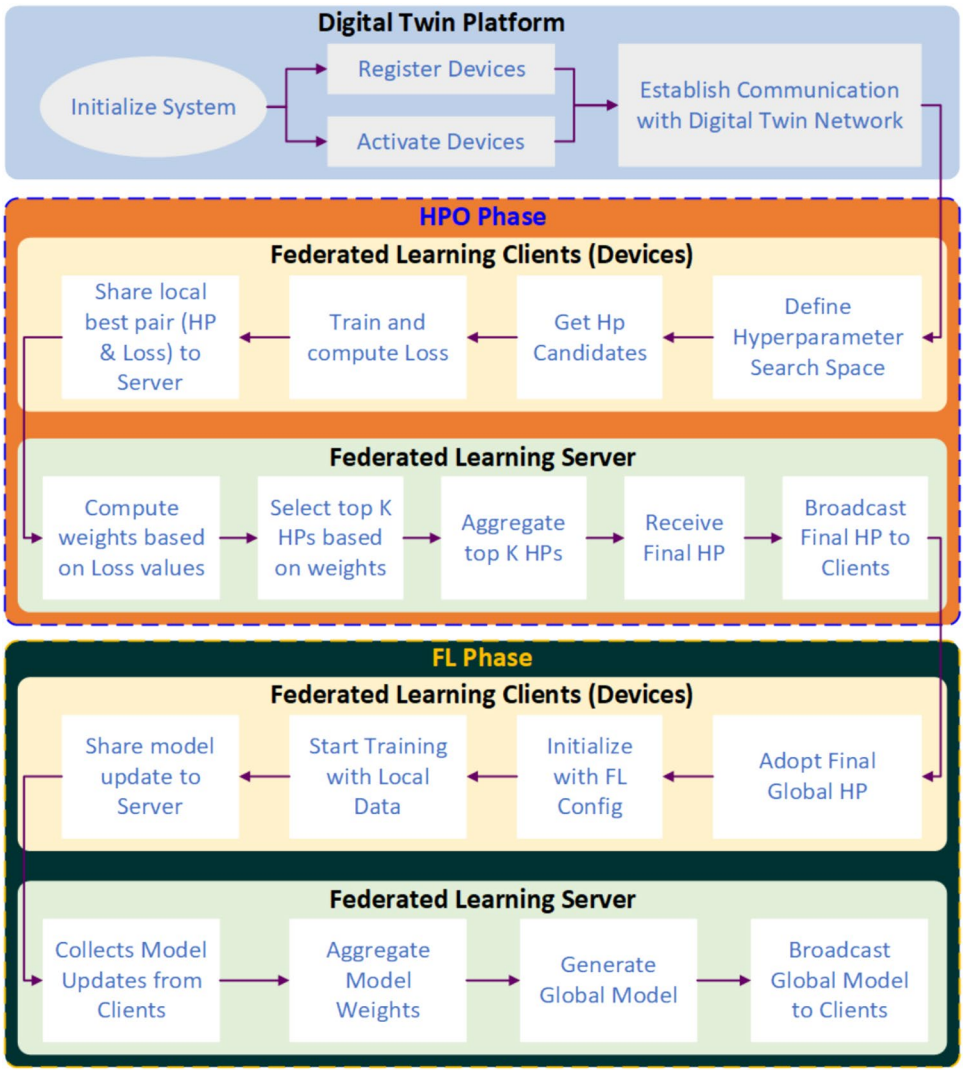


Fig. 3 Layer-wise process flow of weighted hyperparameter aggregation in FL-based DT

set, which is broadcast to all clients. In the FL phase for clients, each adopts the global hyperparameters, initializes the local FL setup, and trains the model on their full local datasets for the specified local epochs. After each local training cycle, clients send only their model updates to the server. In the FL phase on the server, the coordinator combines the received model updates (FedAvg) to create the global model and broadcasts it back to clients for the next training round, completing the HPO to FL pipeline under DT supervision.

Table 1 Detailed development specification of the environment in the DT and federated server

	Feature	Description
Hardware (Desktop)	OS	Windows 10 Pro 64bit
	CPU	Intel®core™ i9-11900F 2.5GHz
	Memory	64 GB
	Storage	1000 GB
Software	Programming Language	Python3.8, HTML5, CSS3, JavaScript
	Application (IDE)	VS Code
	Libraries, and Frameworks	Keras 2.6, TensorFlow 2.6, Flask 2.2.2, Django 4.2.7, NumPy 1.19.5, Requests 2.28
	Database	MySQL Workbench 8.0

Table 2 Detailed development environment of AIoT device

	Feature	Description
Raspberry Pi 4 (Client)	OS	Ubuntu 20.04 64bit
	CPU	Quad-core 64-bit SoC @ 1.5GHz
	Memory	4 GB
	Storage	32 GB
Software	Programming Language	Python3.8
	Application (IDE)	VS Code (Remote SSH)
	Libraries, and Frameworks	Keras 2.6, TensorFlow 2.6, Flask 2.2.2, NumPy 1.19.5, Requests 2.28

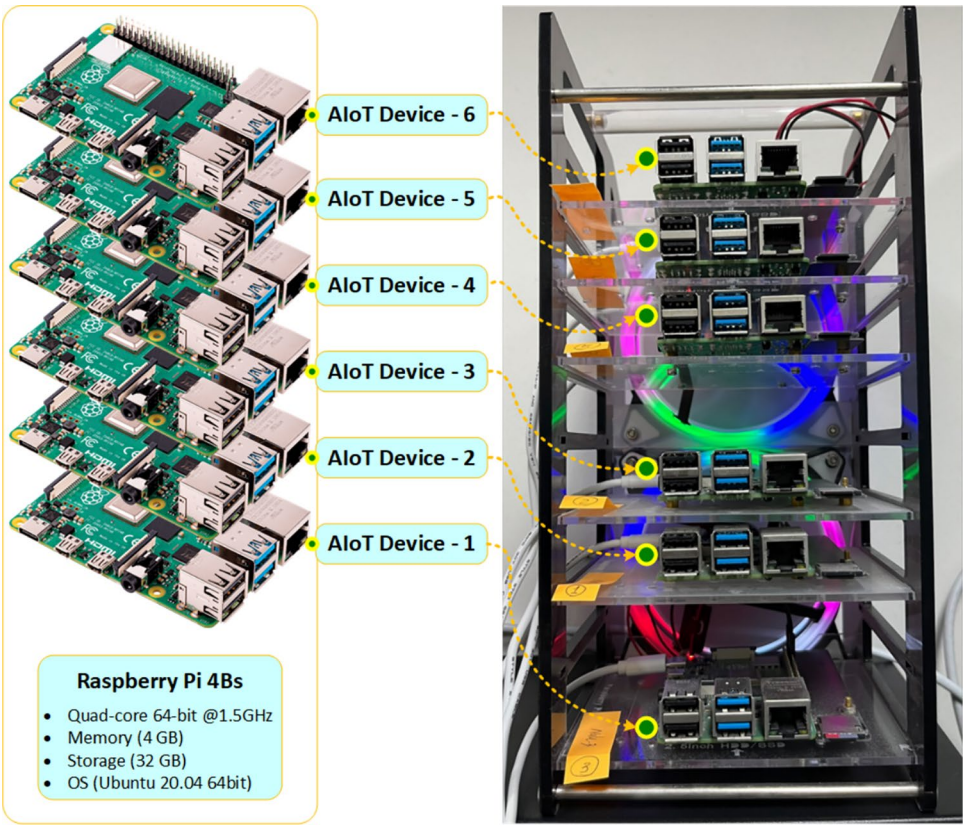


Fig. 4 Physical client devices used to form AIoT network for hyperparameters aggregation in DT-based FL environment

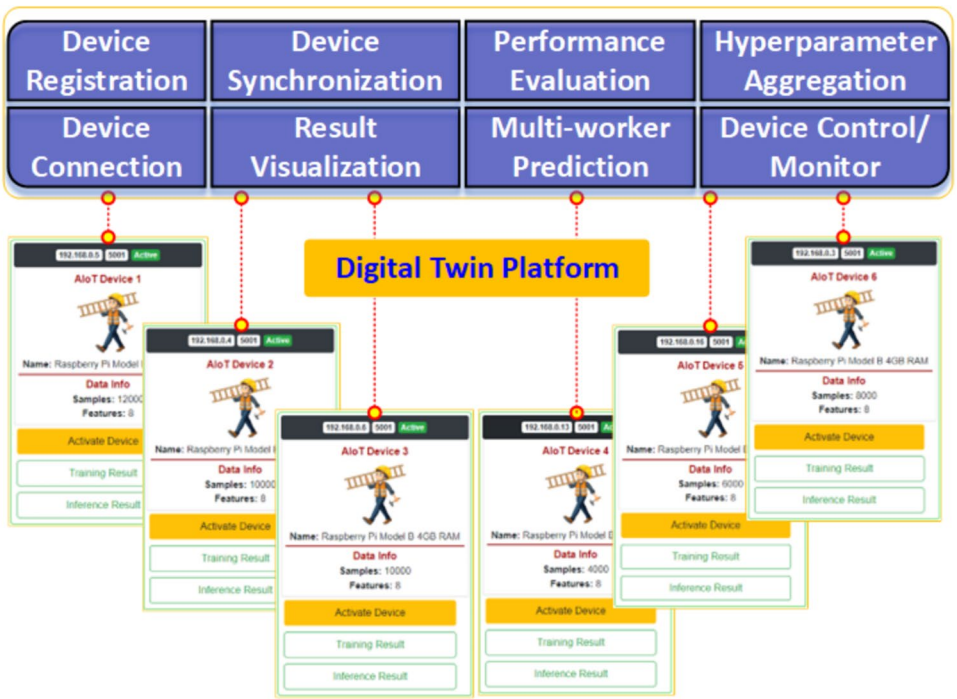


Fig. 5 Virtualization capabilities of the physical devices created in the DT platform

4 Implementation environment and results

The environment in the DT and federated server's comprehensive development specification is displayed in Table 1. A Django framework-based edge server is configured on a high-performance machine to develop an optimal inference model for outdoor construction environments by training data using the DNN learning model and applying the federated learning technique.

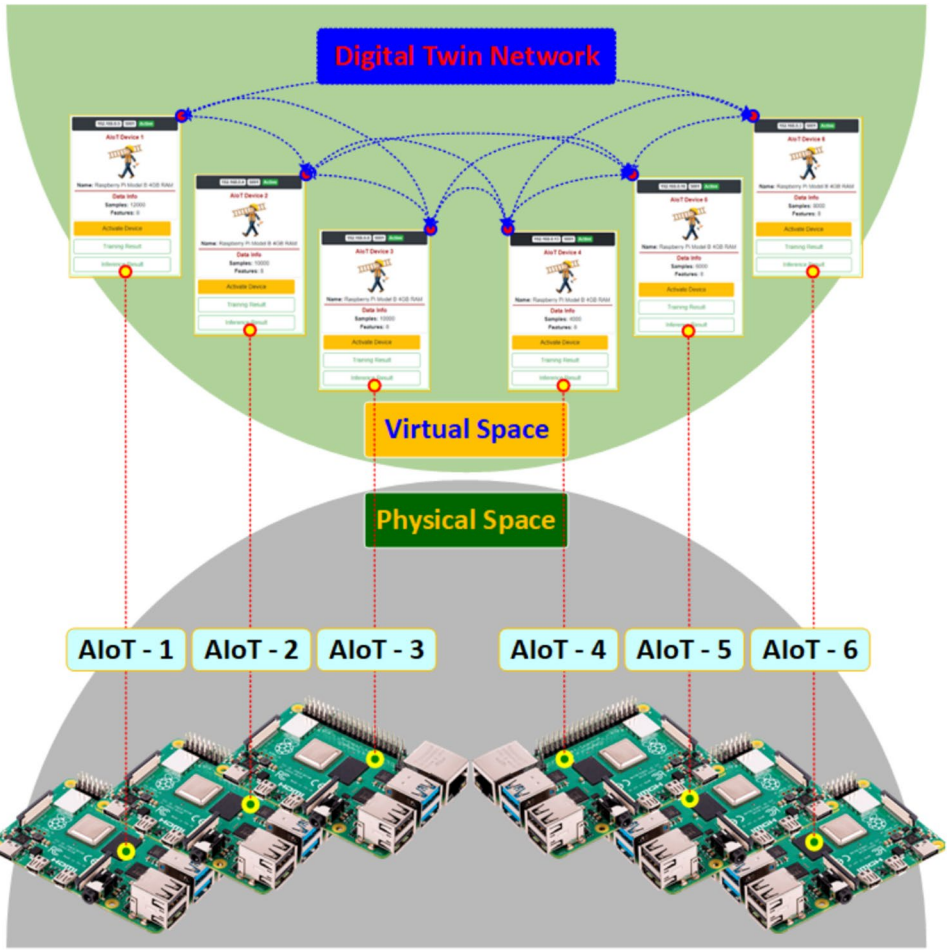


Fig. 6 Virtualization of DT enabling synchronization of physical devices with their virtual twins

The AIoT device's development environment is displayed in Table 2. As an FL client, we use a Raspberry Pi 4 equipped with a quad-core 1.5GHz 64-bit CPU, a 32GB MicroSD card running Ubuntu 20.04 64-bit, and 4GB of RAM. Frameworks such as TensorFlow, Django, and Flask for services that are programmed based on the Python language in the Visual Studio Code (VS Code) IDE. Python version 3.8 serves as the Python interpreter, and it comes pre-installed on the Ubuntu 20.04 OS. Additionally, we have installed the NumPy 1.19.5, requests 2.28, and Keras 2.6 libraries to meet the project requirements.

The client devices utilized in an experiment on hyperparameter aggregation in DT-based federated learning for worker safety are depicted in Fig. 4. Raspberry Pi 4B devices serve as FL clients, which are designed to be intelligent through machine learning, thereby facilitating model training and prediction. These devices are referred to as AIoT due to their capacity to make decisions for specific purposes. Six Raspberry Pi 4B devices are employed in this experiment to establish a testbed for the primary objective of HPO. Subsequently, a worker safety dataset is employed to conduct machine learning tasks. It is essential for the integrity of this research that actual physical devices be employed to provide a genuine experimental experience.

Figure 5 shows six physical Raspberry Pi 4B clients and their six virtual twins. In this work, the DT platform serves as a lightweight control layer around the FL+HPO pipeline rather than a separate research contribution. It provides just the minimal services we needed to run and observe the experiments:

- Device onboarding & status. Register Raspberry Pi clients, record basic metadata (CPU, memory, local dataset size), and show active/inactive state.
- Job orchestration (experiment control). Launch per-client HPO jobs, collect each client's (HP, loss) pairs, run the loss-weighted aggregation on the server module, and broadcast the resulting global hyperparameters to clients; then trigger FL rounds using those settings.
- Basic monitoring. Display per-device and aggregate telemetry needed for the study (validation loss/accuracy or MAE and per-round time) and show train/inference results for each device card.
- Simple UI hooks. Start/stop training or inference from the device cards shown in Fig. 5.

Table 3 Result tables of averaging hyperparameters mechanism

Local Best Candidate Hyperparameters on Six Nodes						
Devices	Activation	Optimizer	Learning_rate	Batch_size	Epochs	Loss
1	tanh	Adam	0.000410	27	10	0.0245
2	relu	Adam	0.011492	54	11	0.0202
3	tanh	Adam	0.000409	49	8	0.0283
4	relu	Adam	0.003446	16	12	0.0213
5	sigmoid	SGD	0.014278	60	9	0.0206
6	relu	SGD	0.003087	22	7	0.0233
Final Global Hyperparameters						
	tanh	Adam	0.005520	38	9	

These functions let us coordinate experiments across heterogeneous, UI-less devices and verify outcomes; a full DT feature set or evaluation is outside our scope and not required for the claims of this paper.

Figure 6 illustrates how the DT platform mirrors six physical AIoT clients (Raspberry Pi 4B, Ubuntu Server, SSH-only) with one-to-one virtual twins. Because these edge devices have no native UI, the DT provides a simple web surface that (i) lists basic hardware/status (device name, CPU, memory), (ii) shows local data summaries (e.g., number of samples/features), and (iii) exposes minimal control actions. From each twin card, an operator can activate/deactivate the device and trigger study-specific tasks such as local HPO, starting/stopping training, and running inference for safety-index prediction. The Digital Twin Network maintains this mapping and relays these lightweight commands and status updates; it does not replace the FL/HPO algorithms but offers a practical access point for coordinating SSH-only devices during experiments.

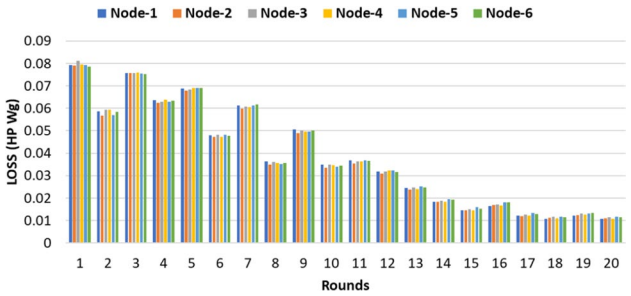


Fig. 7 Loss evaluation using simple hyperparameters averaging mechanism

5 Performance evaluation

5.1 Datasets, model and evaluation metrics

The worker safety dataset, provided by the research center at Jeju National University, consists of 30,000 rows used for training and validation. The data is distributed across six devices, each representing a worker in an outdoor construction environment. The target variable is the worker safety index, with input features including weather (temperature, wind, precipitation), building conditions (strain, inclinometer, load), and worker status (location, heart rate).

For the extended experiments purpose, three publicly available datasets are used: the MNIST [14], EEGEyeState [45], and Weather datasets [24]. The MNIST dataset contains 70,000 grayscale images of handwritten digits (0–9), each 28x28 pixels, commonly used for image classification tasks. The EEGEyeState dataset includes 14,980 EEG signal readings with 14 continuous features, used to classify whether a subject's eyes are open or closed. Additionally, the Weather dataset comprises daily meteorological data, including temperature, humidity, wind speed, and weather conditions, making it suitable for predictive modeling and weather analysis tasks.

Deep Neural Networks (DNNs) [31] are machine learning models that consist of multiple layers of interconnected neurons that enable to learn complex patterns from large datasets. They have been widely used for various tasks, including image recognition, natural language processing, and predictive analytics due to their high accuracy and adaptability. Worker safety is predicted and trained using a DNN model. Two hidden layers with 64 and 32 neurons each, as well as an input layer, are all part of this multilayered neural network. The model considers a number of input factors, including location, inclinometer measurements, wind, temperature, precipitation, load, strain, and heart rate. A safety index, which is produced from 2,689 parameters in total, is the model's output.

Model performance is evaluated using Mean Square Error (MSE) and Mean Absolute Error (MAE) for regression [43], and Accuracy, Binary Crossentropy [11], and Sparse Categorical Crossentropy for classification tasks [7]. MSE (Mean Squared Error) quantifies the average squared difference between actual and predicted values, commonly used in regression tasks:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{6}$$

Table 4 Result tables of weighted hyperparameters aggregation mechanism

Local Best Candidate Hyperparameters on Six Nodes						
Devices	Activation	Optimizer	Learning_rate	Batch_size	Epochs	Loss
1	tanh	Adam	0.001947	59	11	0.0245
2	tanh	Adam	0.013658	27	9	0.0202
3	relu	SGD	0.022289	45	10	0.0223
4	relu	SGD	0.057119	38	11	0.0229
5	tanh	SGD	0.001025	61	8	0.0207
6	relu	Adam	0.007807	17	10	0.0233
Weighted Candidate Hyperparameters Sets						
Top HP	Activation	Optimizer	Learning_rate	Batch_size	Epochs	Loss
1	tanh	Adam	0.013658	27	9	0.0202
2	tanh	SGD	0.001025	61	8	0.0207
3	relu	SGD	0.022289	45	10	0.0223
Final Global Hyperparameters						
	tanh	SGD	0.012324	44	9	

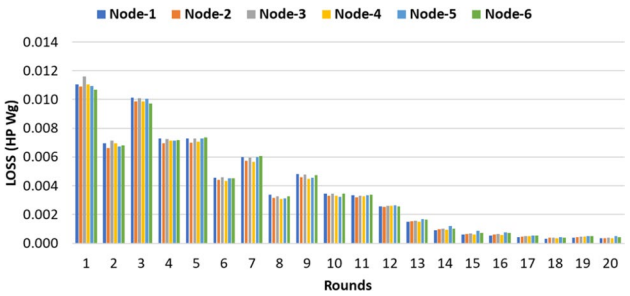


Fig. 8 Loss evaluation using weighted hyperparameters averaging mechanism

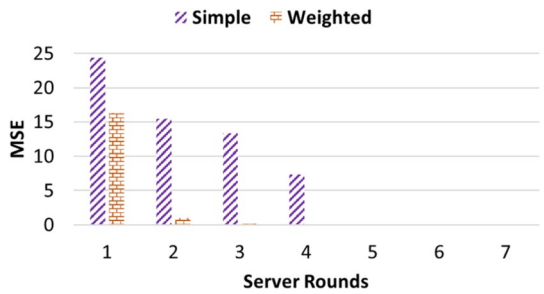


Fig. 9 MAE of the final global model after FL training, comparing simple vs. loss-weighted HP aggregation

MAE (Mean Absolute Error) measures the average absolute difference between actual and predicted values, disregarding the direction of the errors:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{7}$$

Accuracy evaluates classification models by calculating the ratio of correct predictions to the total number of predictions:

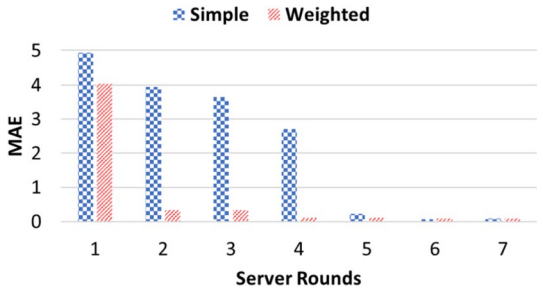


Fig. 10 MSE of the final global model after FL training, comparing simple vs. loss-weighted HP aggregation

$$\text{Accuracy} = \frac{\text{Correct Predictions}}{\text{Total Predictions}} \tag{8}$$

Binary Crossentropy, used in binary classification, compares predicted probabilities against actual binary labels:

$$\text{Binary Crossentropy} = -\frac{1}{n}\sum_{i=1}^{n}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)] \tag{9}$$

Sparse Categorical Crossentropy, applied in multi-class classification, works with integer labels and avoids one-hot encoding, making it more efficient:

$$\text{Sparse Categorical Crossentropy} = -\frac{1}{n}\sum_{i=1}^{n}\log(\hat{y}_{i,y_i}) \tag{10}$$

5.2 Performance evaluation on worker safety dataset

Table 3 presents the results of the simple averaging mechanism. Each node underwent 5 evaluations using the Hyperopt framework for local Bayesian-based HPO. The table details the optimal local hyperparameters optimizer, activation function, learning rate, epochs, and batch size from each

node. The final row shows the aggregated global hyperparameter set, computed by averaging the local sets received by the server. The loss values presented in this table are the validation losses from the HPO phase; they do not represent the ultimate performance metrics for FL.

Figure 7 illustrates the loss evaluation in the collaborative training model on each node over 20 rounds by using the global HP resulting from the simple hyperparameters averaging mechanism. The mean squared error (loss) notably decreases in the initial rounds on each node, but divergence and convergence fluctuate in the following rounds on each node. The fluctuation of divergence and convergence in the training model occurs at each node because all nodes share and train the same model. As the number of rounds increases, the loss steadily decreases.

Table 4 presents the results of the FL weighted hyperparameter aggregation mechanism during the HPO phase. Each node, representing a constrained device monitoring environmental and physiological data (e.g., heart rate, strain, weather conditions), locally optimized hyperparameters such as optimizer, activation function, learning rate, epochs, and batch size are key factors affecting prediction accuracy. After evaluating each node five times using the Hyperopt framework with Bayesian-based HPO, the top three hyperparameter sets, ranked by loss, were selected. The server aggregated these sets through weighted averaging to produce the final global hyperparameter set, which is then deployed across all nodes for the FL phase, thereby improving the global model's performance in predicting safety hazards. Loss values in this table are HPO-phase validation losses used to weight candidates; they are not the final FL performance metrics.

Figure 8 illustrates the evaluation of loss in the collaborative training model on each node over 20 rounds by using the global HP resulting from the Weighted Hyperparameters Aggregation Mechanism. The mean squared error (loss) significantly decreases in the initial rounds on each node, but there is a fluctuation between divergence and convergence in the following rounds on each node. As the number of rounds increases, the loss slowly decreases.

Figure 9 compares the MAE of the global model using Averaged and Weighted Hyperparameter Aggregation mechanisms. The Averaged HPO shows slow convergence, requiring more rounds to stabilize, indicating suboptimal hyperparameter selection. In contrast, the Weighted HPO converges faster and identifies optimal hyperparameters. From the second round onward, Weighted HPO consistently achieves lower error, enhancing the model's accuracy in predicting safety hazards, improving its relevance and performance in the worker safety use case.

Figure 10 compares the MSE for the global model using Averaged and Weighted Hyperparameter Aggregation mechanisms across seven server rounds in the worker safety prediction task. Accurate and fast convergence is critical in predicting safety risks, as delays or inaccuracies may lead to dangerous conditions. The Averaged HPO mechanism exhibits slow convergence, stabilizing after five rounds, suggesting suboptimal hyperparameter selection for the safety task. In contrast, the Weighted HPO mechanism converges within one round, efficiently identifying optimal hyperparameters. This rapid convergence and improved predictive performance are essential for timely and precise safety hazard predictions. The Weighted HPO consistently achieves lower error from the second round onward, enhancing the model's predictive accuracy and ensuring effective real-time risk mitigation in dynamic worker safety environments.

Table 5 Performance Comparison between Averaged and Weighted Mechanisms in IID setting

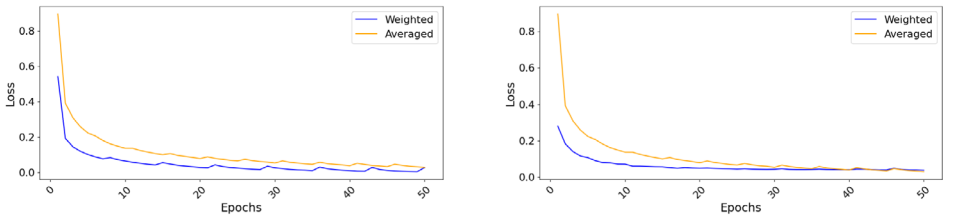
Dataset	Domain	Metric	Clients	Simple-AVG	Weighted-AVG
EEG-EyeState	Tabular	Accuracy	6	0.9695	0.9847
			10	0.9594	0.9738
			15	0.9469	0.9653
MNIST	CV	Accuracy	6	0.9691	0.982
			10	0.9491	0.9544
			15	0.9361	0.9517
Weather	Tabular	MAE	6	0.1312	0.1228
			10	0.1374	0.1285
			15	0.1426	0.1305
Safety	Tabular	MAE	6	0.0252	0.0231
			10	0.0325	0.0264
			15	0.0391	0.0309

Table 6 Performance Comparison between Averaged and Weighted Mechanisms in non-IID setting

Dataset	Domain	Metric	Clients	Simple-AVG	Weighted-AVG
EEG-EyeState	Tabular	Accuracy	6	0.8846	0.8925
			10	0.8418	0.8509
			15	0.7379	0.8055
MNIST	CV	Accuracy	6	0.9195	0.9254
			10	0.9029	0.9197
			15	0.8987	0.9027
Weather	Tabular	MAE	6	0.1889	0.1753
			10	0.2086	0.1821
			15	0.2389	0.2178
Safety	Tabular	MAE	6	0.0562	0.0482
			10	0.0611	0.0571
			15	0.0709	0.0689

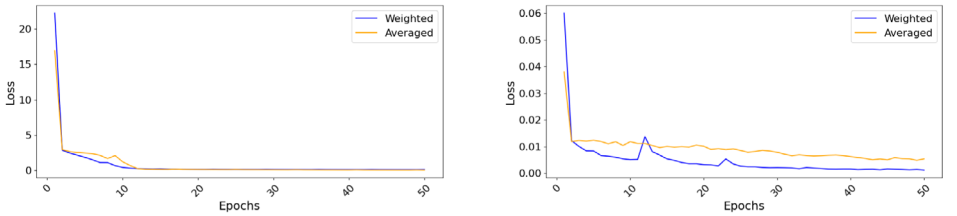
5.3 Empirical analysis

In order to assess the effectiveness of our proposed method, we conducted extensive experiments using public datasets as case studies. We utilized the EEGEyeState, MNIST, and Weather datasets to provide a balanced assessment



(a) Training Loss Comparison on the MNIST Dataset

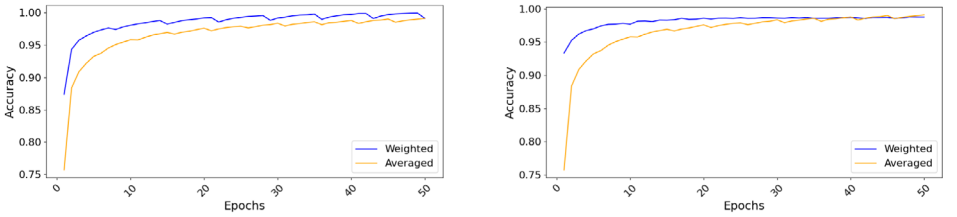
(b) Training Loss Comparison on the EEGEyeState Dataset



(c) Training Loss Comparison on Weather Dataset

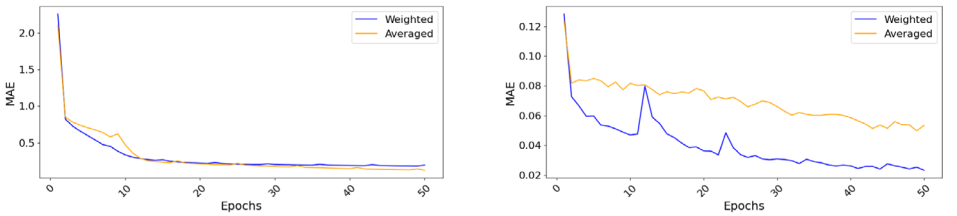
(d) Training Loss Comparison on Safety Dataset

Fig. 11 Loss comparison between averaged and weighted hyperparameters aggregation mechanisms



(a) Accuracy Comparison on the MNIST Dataset

(b) Accuracy Comparison on the EEGEyeState Dataset



(c) MAE Comparison on Weather Dataset

(d) MAE Comparison on Safety Dataset

Fig. 12 Accuracy and MAE-based comparative evaluation of averaged and weighted hyperparameters aggregation mechanisms

across different machine-learning problems. Specifically, we focused on two datasets (MNIST and EEGEyeState) for classification and two datasets (Weather and Safety) for regression. Tables 5 and 6 present a comprehensive comparison of the Averaged HP and Weighted HP aggregation mechanisms in both IID and non-IID scenarios. To simulate various data heterogeneity settings, we employed the Dirichlet distribution method [44], with the degree of heterogeneity controlled by $\alpha = 0.5$. Our experiments encompassed simulations with different numbers of clients (6, 10, and 15) to assess scalability within the constrained environment. All evaluations were conducted on the actual testbed depicted in Figure 4. The results demonstrate a clear correlation between the number of participating clients and model performance: as the client count increases, we observe a corresponding decrease in model performance. This highlights the challenges of maintaining model quality in larger federated learning networks, particularly under non-IID conditions.

Unless noted otherwise, results are from a single end-to-end run per setting (fixed random seed controlling Dirichlet partition, initialization, and HPO proposal order) to reflect the behavior on our constrained testbed.

Figures 11 and 12 display the loss, accuracy, and MAE assessments for each dataset in the IID setting. Specifically, Fig. 11(a) through (d) compare the loss evaluation between the Averaged and Weighted mechanisms, while Fig. 12(a) through (d) compare the accuracy and MAE for three public datasets (MNIST, EEGEyeState, and Weather) and the private Worker Safety dataset. Figure 11(a), using the MNIST

Table 7 Comparison of hyperparameter aggregation strategies (10 clients)

Dataset	Part.	Metric	Simple-AVG	ServerSingleShot†	**Weighted (ours)**
EEG-EyeState	IID	Acc	0.9594	0.9663	**0.9738**
EEG-EyeState	non-IID	Acc	0.8418	0.8512	**0.8509**
MNIST	IID	Acc	0.9491	0.9511	**0.9544**
MNIST	non-IID	Acc	0.9029	0.9085	**0.9197**
Weather	IID	MAE	0.1374	0.1317	**0.1285**
Weather	non-IID	MAE	0.2086	0.1964	**0.1821**
Safety	IID	MAE	0.0325	0.0281	**0.0264**
Safety	non-IID	MAE	0.0611	0.0595	**0.0571**

†FLoRa-style proxy: server picks the single client-proposed HP with the lowest HPO loss; no Top-*K*, no weighting, no model sharing

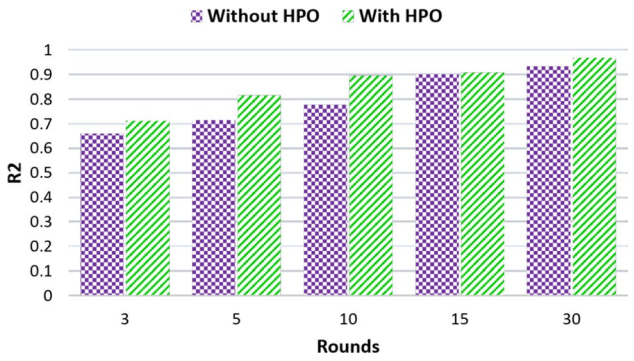


Fig. 13 R2 comparison between FL with HPO and FL without HPO

dataset, shows that the Weighted mechanism converges faster and achieves lower loss values compared to the Averaged mechanism. Similarly, Figure 11(b), based on the EEGEyeState dataset, reveals that the Weighted mechanism achieves lower loss values and faster convergence, whereas the Averaged mechanism reaches similar loss levels at later epochs. In Figure 11(c), the Weather dataset results highlight that while both mechanisms achieve similar final loss values, the Weighted mechanism stabilizes more quickly. Lastly, Figure 11(d), which uses the private Worker Safety dataset, indicates that the Weighted aggregation leads to improved convergence behavior with slightly higher peaks in fluctuations but ultimately lower loss compared to the Averaged aggregation.

The accuracy results align with the loss comparisons. Figure 12(a), using the MNIST dataset, demonstrates that the Weighted mechanism achieves higher accuracy more quickly compared to the Averaged mechanism. Similarly, Figure 12(b), based on the EEGEyeState dataset, shows that the Weighted mechanism reaches a higher and more stable accuracy, while the Averaged mechanism exhibits more significant fluctuations. The MAE comparisons follow a similar trend. Figure 12(c), for the Weather dataset, shows that both mechanisms achieve similar final MAE values, but the Weighted mechanism converges and stabilizes faster. Finally, Figure 12(d), using the private Worker Safety dataset, reveals that the Weighted aggregation achieves lower MAE values with fewer but slightly more pronounced fluctuations compared to the Averaged aggregation.

Across all datasets, the *Weighted* curve (blue) shows:

- consistently lower training loss than *Simple-AVG* (orange) throughout training (Fig. 11a–d);
- faster rise to high accuracy on MNIST/EEGEyeState (Fig. 12a–b), with fewer early-epoch oscillations; and
- lower/stabler MAE on Weather and Safety (Fig. 12c–d).

These plots illustrate the trajectory-level effect (earlier stabilization and better end values) under IID; complementary non-IID outcomes are summarized numerically in Tables 5 and 6.

Despite the IID nature of the datasets, fluctuations during training are observed in both mechanisms. This behavior arises from the aggregation process in FL, where model parameters from all clients are averaged during each round, which can introduce variability in the updates. The observed fluctuations during training can be attributed to the inherent dynamics of FL, where aggregated updates from distributed clients are averaged during each round. This process introduces variability, particularly in the early epochs, as clients have different data distributions and optimization progress. Fluctuations primarily occur due to heterogeneous dataset sizes, varying local convergence rates, and client-specific optimization landscapes. Clients with smaller datasets may converge faster, while those with larger datasets contribute more substantial updates, leading to inconsistencies in global model aggregation. Additionally, the method of hyperparameter averaging (simple vs. weighted) impacts stability, as weighted averaging prioritizes clients differently, influencing convergence speed. To address fluctuations in federated learning, client selection based on performance during model updates can prioritize reliable participants, while clustered aggregation groups clients with similar data distributions to stabilize training. Excluding underperforming clients could further enhance robustness, even in the presence of heterogeneous and non-IID data.

Table 7 presents a head-to-head performance comparison across four datasets under both IID and non-IID partitions. We evaluate our Weighted top-K aggregation method against two baselines: Simple-AVG (unweighted averaging) and ServerSingleShot, a FLoRa-style proxy that selects the single client hyperparameter set with the lowest local

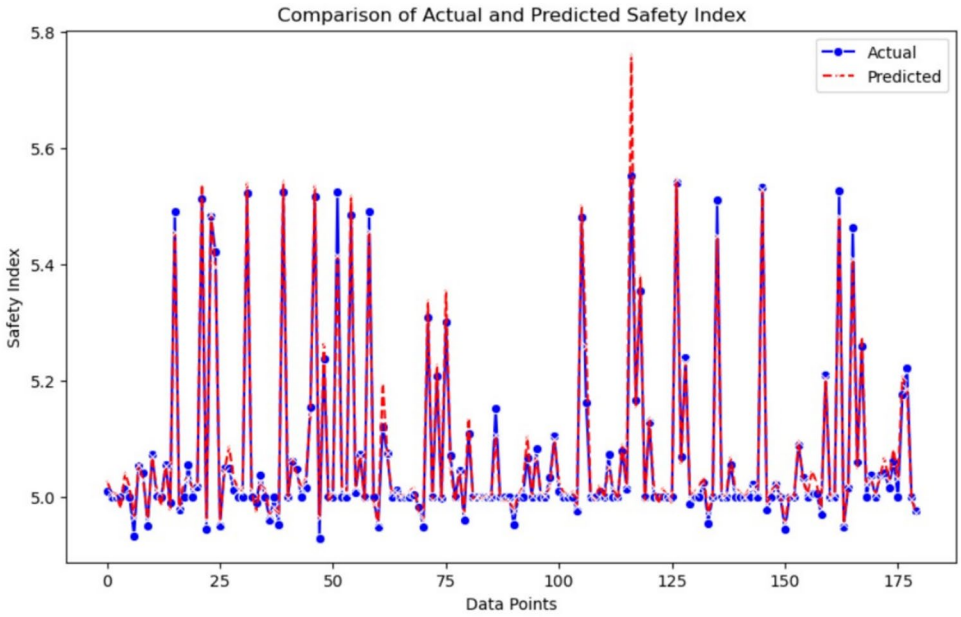


Fig. 14 Comparison between the actual and predicted safety index of the worker on a device

Table 8 Per-device RTT (ms) for on-device safety-index inference

Device	Min	Median	Mean	Max
Pi#1	342	361	372	458
Pi#2	340	365	370	450
Pi#3	338	363	368	445
Pi#4	340	364	371	447
Pi#5	343	368	378	453
Pi#6	335	360	365	435

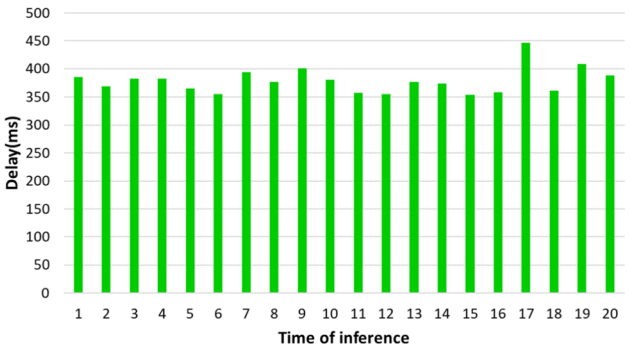


Fig. 15 RTT (ms) across 20 consecutive inference samples per device class

validation loss. Our proposed weighted approach consistently outperforms the baselines, achieving a measurable "Gain vs. Simple" through increased classification accuracy and decreased Mean Absolute Error (MAE). These results demonstrate the robustness of loss-based weighting for hyperparameter aggregation in federated settings.

Furthermore, to evaluate the robustness of the global model resulting from the proposed approach towards worker safety, we assessed the performance of the inference model for predicting worker safety using real-time unseen data. Figure 13 depicts the R2 assessment comparison between FL without HPO and with HPO. The results demonstrate that FL with HPO consistently outperforms FL without HPO across most rounds of global iteration, indicating that the proposed method effectively enhances worker safety.

5.4 Worker safety monitoring result and discussion

The increasing urban population, expanding by 200,000 individuals per day, is exerting a profound influence on the global construction industry [1]. As cities grow and construction projects surge, ensuring worker safety becomes more challenging due to the dynamic and intricate nature of construction sites. Relying solely on physical monitoring can lead to misleading assessments, especially under fast-changing on-site conditions. In response, decentralized training (FL with HPO) enables continuous safety monitoring and adaptive responses without constant physical observation.

To demonstrate this approach, we conducted a case study aimed at predicting and monitoring worker safety. Utilizing a globally trained model, optimized with the globally tuned hyperparameter (HP) set derived through a weighted aggregation method, we simulated a one-hour-ahead prediction of a worker's safety status. Data for inference was gathered at 20-second intervals, resulting in 180 total observations. The comparison between the actual and predicted safety indices, as illustrated in Figure 14, highlights the model's performance across the collected data points.

The global model runs *on-device* and outputs a safety index every 20 s. The DT page only displays per-worker indices and relays basic notifications; inference and thresholding occur on the device.

(i) *Wearables.* If the index falls below a threshold τ or its rate of change exceeds ρ, the device triggers a brief haptic/audio alert (with a 30-second cooldown to avoid fatigue).
(ii) *Supervisor view.* The DT page consolidates indices, highlights the lowest-K workers, and displays contributing signals (e.g., wind/strain spikes, elevated heart) rate).
(iii) *Escalation.* If a low index persists for T minutes or multiple nearby workers fall below au, the DT sends a high-priority alert (via SMS/app) to the supervisor.

For this study, we used a single global threshold τ per target and a 30 s cooldown; policy tuning is site-specific and outside our scope.

Table 8 reports per-device round-trip time (RTT) statistics corresponding to Fig. 15. Median RTTs are ≤ 368 ms on all nodes, with maximum RTT ≤ 458 ms, supporting near–real-time alerts at a 20 s sampling interval.

Latency depends on (a) model size and layer types; (b) input window length and feature count; (c) batch size (we use batch=1 on the device); (d) preprocessing costs (e.g., normalization); and (e) device load and runtime libraries (e.g., NEON, threading). In our setup, we keep architecture, windowing, and batch size constant across methods, so differences in RTT mainly result from system noise rather than architectural changes. Optimizations like quantization or operator fusion are separate considerations and will be addressed in future work.

Section 5 compares *Weighted* and *Simple* aggregation across datasets and IID/non-IID conditions (Tables 5, 6, Figs. 11, 12); Figure 13 further contrasts FL with and without HPO for the safety case. Additional comparisons (e.g., different architectures or quantized variants) would mainly examine inference-time trade-offs rather than the proposed HP aggregation itself and are noted as future work.

6 Conclusions

In conclusion, hyperparameters play a crucial role in the machine learning process, influencing the model's learning and performance. While centralized machine learning has extensively explored HPO, FL presents unique challenges for HPO due to the decentralized nature of the data. The limitations of conventional HPO approaches in FL settings stem from the need for more access to the complete dataset and the high communication and training costs associated with training numerous models for various hyper-parameter configurations. One-shot local Bayesian Optimization is used to identify optimal hyperparameters for each client using a fraction of the dataset. Initially, FL clients determine and transmit their optimal hyperparameters to a central server, which uses a weighted consolidation strategy to create and distribute a global hyperparameter set. This method improves model performance and convergence, ensuring scalable federated learning deployments in worker safety-critical applications. Across different datasets, client distributions, and evaluation metrics (accuracy for classification and MAE for regression), the weighted hyperparameter consolidation method demonstrates an overall improvement of approximately 6.44% across all datasets and client counts (increased accuracy for classification; decreased MAE for regression, each normalized and averaged) compared to the simple averaging approach. Despite the significance of HPO in FL, there remains ample opportunity for further exploration and application across diverse domains. Our study, which mainly focused on worker safety data, underscores the importance of addressing these challenges to maximize the benefits of FLHPO in real-world scenarios.

The proposed mechanism enhances FL by optimizing hyperparameters through a weighted consolidation strategy, improving model performance and scalability in worker safety applications. However, privacy risks arising from exchanging unprotected hyperparameters highlight the need for approaches that safeguard information while enabling collaborative tuning. Additionally, addressing communication overhead in large-scale networks and benchmarking against advanced HPO methods remain key areas for future research to maximize FLHPO's real-world impact.

Limitations and future work. Our evaluation involves up to 15 clients on Raspberry Pi–class devices; behavior at larger scales still needs investigation. The loss-weighted aggregation assumes honest, fairly calibrated client validation losses, and although only hyperparameters (not model weights) are shared, some basic information leakage is possible. We present results from a single seed per setting and focus on non-visual, tabular, and temporal signals. Vision tasks (e.g., PPE detection) and additional baselines beyond Simple-AVG and a server single-shot proxy are planned for future work. Moving forward, we will explore larger client populations (such as partial participation and lightweight multi-fidelity search), improve robustness against noisy or misreported losses (e.g., with robust weighting and secure aggregation), and combine the method with lightweight vision models.

Acknowledgements This research was supported by the Regional Innovation System & Education (RISE) program through the Jeju RISE center, funded by the Ministry of Education (MOE) and the Jeju Special Self-Governing Province, Republic of Korea (2026-RISE-17-001), this work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00405278), and this work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (No. RS-2024-00346238). Any correspondence

related to this paper should be addressed to DoHyeun Kim.

Author contributions Conceptualization, S.M., and A.-N.K.; validation, S.M., and A.T.N.; investigation, S.M., and A.-N.K.; resources, A.-N.K., and D.-H.K.; data curation, S.M., and A.T.N.; writing—original draft preparation, S.M.; writing—review and editing, A.-N.K., and D.-H.K.; supervision, D.-H.K.; project administration, D.-H.K.; All authors have read and agreed to the published version of the manuscript.

Data availability No datasets were generated or analysed during the current study.

Declarations

Competing interests The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. 68% of the world population projected to live in urban areas by 2050, says UN. Available: https://www.un.org/development/desa/en/news/population/2018-revision-of-world-urbanization-prospects.html (2018). Accessed 22 Oct 2024 [Online]
2. Adriaensen, S., Biedenkapp, A., Shala, G., et al.: Automated dynamic algorithm configuration. Journal of Artificial Intelligence Research **75**, 1633–1699 (2022)
3. Agrawal, S., Sarkar, S., Alazab, M., et al.: Genetic cfl: hyperparameter optimization in clustered federated learning. Comput. Intell. Neurosci. **2021**(1), 7156420 (2021)
4. Akanmu, A.A., Anumba, C.J., Ogunseiju, O.O.: Towards next generation cyber-physical systems and digital twins for construction. Journal of Information Technology in Construction (2021). https://doi.org/10.36680/j.itcon.2021.027
5. Alibrahim, H., Ludwig, S.A.: Hyperparameter optimization: Comparing genetic algorithm against grid search and bayesian optimization. In: 2021 IEEE Congress on Evolutionary Computation (CEC), IEEE, pp 1551–1559 (2021)
6. Alnowaiser, K.K., Ahmed, M.A.: Digital twin: current research trends and future directions. Arab. J. Sci. Eng. **48**(2), 1075–1095 (2023)
7. Andrei-Alexandru, T., Henrietta, D.E.: Low cost defect detection using a deep convolutional neural network. In: 2020 IEEE International conference on automation, quality and testing, robotics (AQTR), IEEE, pp 1–5 (2020)
8. Anggoro, D.A., Mukti, S.S.: Performance comparison of grid search and random search methods for hyperparameter tuning in extreme gradient boosting algorithm to predict chronic kidney failure. International Journal of Intelligent Engineering & Systems. **14**(6), (2021)
9. Arango, S.P., Jomaa, H.S., Wistuba, M., et al.: Hpo-b: A large-scale reproducible benchmark for black-box hpo based on openml. (2021). arXiv preprint arXiv:2106.06257
10. Baik, S., Choi, M., Choi, J., et al.: Meta-learning with adaptive hyperparameters. Adv. Neural. Inf. Process. Syst. **33**, 20755–20765 (2020)
11. Brahmbhatt, P., Rajan, S.N.: Skin lesion segmentation using segnet with binary crossentropy. In: Proceedings of the International Conference on Artificial Intelligence and Speech Technology (AIST2019), Delhi, India, pp 14–15 (2019)
12. Charles, Z., Garrett, Z., Huo, Z., et al.: On large-cohort training for federated learning. Adv. Neural. Inf. Process. Syst. **34**, 20461–20475 (2021)
13. Dai, Z., Low, B.K.H., Jaillet, P.: Federated bayesian optimization via thompson sampling. Adv. Neural. Inf. Process. Syst. **33**, 9687–9699 (2020)
14. Deng, L.: The mnist database of handwritten digit images for machine learning research. IEEE Signal Process. Mag. **29**(6), 141–142 (2012)
15. Eggensperger, K., Müller, P., Mallik, N., et al.: Hpobench: A collection of reproducible multi-fidelity benchmark problems for hpo. (2021). arXiv preprint arXiv:2109.06716
16. Falkner, S., Klein, A., Hutter, F.: Bohb: Robust and efficient hyperparameter optimization at scale. In: International conference on machine learning, PMLR, pp 1437–1446 (2018)
17. Fang, Q., Chen, X., Castro-Lacouture, D., et al.: Intervention and management of construction workers' unsafe behavior: a simulation digital twin model. Adv. Eng. Inform. **58**, 102182 (2023)
18. Florea, A.C., Andonie, R.: Weighted random search for hyperparameter optimization. (2020). arXiv preprint arXiv:2004.01628
19. Guo, P., Yang, D., Hatamizadeh, A., et al.: Auto-fedrl: Federated hyperparameter optimization for multi-institutional medical image segmentation. In: European Conference on Computer Vision, Springer, pp 437–455 (2022)
20. Hansen, N., Auger, A., Ros, R., et al.: Coco: a platform for comparing continuous optimizers in a black-box setting. Optimization Methods and Software **36**(1), 114–144 (2021)
21. Hossain, M.I., Hosen, M.M., Sunny, M.A.U., et al.: Implementing advanced technologies for enhanced construction site safety. American Journal of Advanced Technology and Engineering Solutions **1**(02), 01–31 (2025)
22. Hutter, F., Hoos, H.H., Leyton-Brown, K.: Sequential model-based optimization for general algorithm configuration. In: Learning and Intelligent Optimization: 5th International Conference, LION 5, Rome, Italy, January 17-21, 2011. Selected Papers 5, Springer, pp 507–523 (2011)
23. Jamieson, K., Talwalkar, A.: Non-stochastic best arm identification and hyperparameter optimization. In: Artificial intelligence and statistics, PMLR, pp 240–248 (2016)
24. Jothi, M.: Weather dataset. (2023). https://www.kaggle.com/datasets/muthuj7/weather-dataset. Accessed 22 Oct 2024
25. Khan, S., Rizwan, A., Khan, A.N., et al.: A multi-perspective revisit to the optimization methods of neural architecture search and hyper-parameter optimization for non-federated and federated learning environments. Computers and Electrical Engineering **110**, 108867 (2023)
26. Khodak, M., Li, T., Li, L., et al.: Weight sharing for hyperparameter optimization in federated learning. In: Int. Workshop on Federated Learning for User Privacy and Data Confidentiality in Conjunction with ICML (2020)
27. Khodak, M., Tu, R., Li, T., et al.: Federated hyperparameter tuning: Challenges, baselines, and connections to weight-sharing. In: Ranzato M, Beygelzimer A, Dauphin Y, et al (eds) Advances in

Neural Information Processing Systems, vol 34. Curran Associates, Inc., pp 19184–19197 (2021)
28. Khodak, M., Tu, R., Li, T., et al.: Federated hyperparameter tuning: challenges, baselines, and connections to weight-sharing. Adv. Neural. Inf. Process. Syst. **34**, 19184–19197 (2021)
29. Kim, J.Y., Cho, S.B.: Evolutionary optimization of hyperparameters in deep learning models. In: 2019 ieee congress on evolutionary computation (cec), IEEE, pp 831–837 (2019)
30. Kundroo, M., Kim, T.: Federated learning with hyper-parameter optimization. Journal of King Saud University-Computer and Information Sciences **35**(9), 101740 (2023)
31. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. nature **521**(7553), 436–444 (2015)
32. Lee, H.B., Lee, H., Shin, J., et al.: Online hyperparameter meta-learning with hypergradient distillation. (2021). arXiv preprint arXiv:2110.02508
33. Li, L., Jamieson, K., DeSalvo, G., et al.: Hyperband: a novel bandit-based approach to hyperparameter optimization. J. Mach. Learn. Res. **18**(185), 1–52 (2018)
34. Li, X., Zeng, J., Chen, C., et al.: Decentralized adaptive work package learning for personalized and privacy-preserving occupational health and safety monitoring in construction. Autom. Constr. **165**, 105556 (2024)
35. Li, Z., Li, H., Zhang, M.: Hyper-parameter tuning of federated learning based on particle swarm optimization. In: 2021 IEEE 7th International Conference on Cloud Computing and Intelligent Systems (CCIS), IEEE, pp 99–103 (2021)
36. Lindauer, M., Eggensperger, K., Feurer, M., et al.: Smac3: a versatile bayesian optimization package for hyperparameter optimization. J. Mach. Learn. Res. **23**(54), 1–9 (2022)
37. Liu, Z.S., Meng, X.T., Xing, Z.Z., et al.: Digital twin-based intelligent safety risks prediction of prefabricated construction hoisting. Sustainability **14**(9), 5179 (2022)
38. McMahan, B., Moore, E., Ramage, D., et al.: Communication-efficient learning of deep networks from decentralized data. In: Artificial intelligence and statistics, PMLR, pp 1273–1282 (2017)
39. Mitic, N., Pyrgelis, A., Sav, S.: How to privately tune hyperparameters in federated learning? insights from a benchmark study. (2024). arXiv preprint arXiv:2402.16087
40. Nnaji, C., Awolusi, I., Park, J., et al.: Wearable sensing devices: towards the development of a personalized system for construction safety and health risk mitigation. Sensors **21**(3), 682 (2021)
41. Nozari, H., Ghahremani-Nahr, J., Szmelter-Jarosz, A.: Chapter one - ai and machine learning for real-world problems. In: Kim S, Deka GC (eds) Artificial Intelligence and Machine Learning for Open-world Novelty, Advances in Computers, vol 134. Elsevier, p 1–12. (2024).https://doi.org/10.1016/bs.adcom.2023.02.001
42. Ozdayi, M.S., Kantarcioglu, M., Gel, Y.R.: Defending against backdoors in federated learning with robust learning rate. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 9268–9276 (2021)
43. Rainio, O., Teuho, J., Klén, R.: Evaluation metrics and statistical tests for machine learning. Sci. Rep. **14**(1), 6086 (2024)
44. Reguieg, H., El Hanjri, M., El Kamili, M., et al.: A comparative evaluation of fedavg and per-fedavg algorithms for dirichlet distributed heterogeneous data. In: 2023 10th International Conference on Wireless Networks and Mobile Communications (WINCOM), IEEE, pp 1–6 (2023)
45. Roesler, O.: EEG eye state. UCI Machine Learning Repository (2013). https://doi.org/10.24432/C57G7J
46. Saadati, Y., Amini, M.H.: Hyper-parameter optimization for federated learning with step-wise adaptive mechanism. (2024). arXiv preprint arXiv:2411.12244
47. Sacks, R., Brilakis, I., Pikas, E., et al.: Construction with digital twin information systems. Data-Centric Engineering **1**, e14 (2020)
48. Shahriari, B., Swersky, K., Wang, Z., et al.: Taking the human out of the loop: a review of bayesian optimization. Proc. IEEE **104**(1), 148–175 (2015)
49. Siddiqi, S., Qureshi, F., Lindstaedt, S., et al.: Detecting outliers in non-iid data: a systematic literature review. IEEE Access **11**, 70333–70352 (2023). https://doi.org/10.1109/access.2023.3294096
50. da Silva, C.N., Prazeres, C.V.: Tiny federated learning for constrained sensors: A systematic literature review. IEEE Sensors Reviews (2025)
51. Snoek, J., Larochelle, H., Adams, R.P.: Practical bayesian optimization of machine learning algorithms. Advances in neural information processing systems 25 (2012)
52. Storn, R., Price, K.: Differential evolution-a simple and efficient heuristic for global optimization over continuous spaces. Journal of Global Optimization **11**, 341–359 (1997)
53. Thelen, A., Zhang, X., Fink, O., et al.: A comprehensive review of digital twin-part 1: modeling and twinning enabling technologies. Structural and Multidisciplinary Optimization **65**(12), 354 (2022)
54. Turner, R., Eriksson, D., McCourt, M., et al.: Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020. In: NeurIPS 2020 Competition and Demonstration Track, PMLR, pp 3–26 (2021)
55. Victoria, A.H., Maragatham, G.: Automatic tuning of hyperparameters using bayesian optimization. Evol. Syst. **12**(1), 217–223 (2021)
56. Wang, Z., Song, M., Zhang, Z., et al.: Beyond inferring class representatives: User-level privacy leakage from federated learning. In: IEEE INFOCOM 2019-IEEE conference on computer communications, IEEE, pp 2512–2520 (2019)
57. Weerts, H.J., Mueller, A.C., Vanschoren, J.: Importance of tuning hyperparameters of machine learning algorithms. (2020). arXiv preprint arXiv:2007.07588
58. Yuan, J., Xu, M., Zhao, Y., et al.: Federated neural architecture search. (2020). arXiv preprint arXiv:2002.06352
59. Zawad, S., Ma, X., Yi, J., et al.: Fedcust: offloading hyperparameter customization for federated learning. Performance Evaluation **167**, 102450 (2025)
60. Zhang, H., Zhang, M., Liu, X., et al.: Fedtune: Automatic tuning of federated learning hyper-parameters from system perspective. In: MILCOM 2022-2022 IEEE Military Communications Conference (MILCOM), IEEE, pp 478–483 (2022)
61. Zhang, H., Fu, L., Zhang, M., et al.: Federated learning hyperparameter tuning from a system perspective. IEEE Internet of Things Journal **10**(16), 14102–14113 (2023). https://doi.org/10.1109/jiot.2023.3253813
62. Zhang, X., Fu, L., Zhang, H., et al.: Federated learning hyperparameter tuning for edge computing. In: Edge Computing-Technology, Management and Integration. IntechOpen (2023)
63. Zhao, B., Mopuri, K.R., Bilen, H.: idlg: Improved deep leakage from gradients. (2020). arXiv preprint arXiv:2001.02610
64. Zhao, Z., Shen, L., Yang, C., et al.: Iot and digital twin enabled smart tracking for safety management. Computers & Operations Research **128**, 105183 (2021)
65. Zhou, Y., Ram, P., Salonidis, T., et al.: Flora: Single-shot hyperparameter optimization for federated learning. (2021). arXiv preprint arXiv:2112.08524
66. Zhu, H., Jin, Y.: Real-time federated evolutionary neural architecture search. IEEE Trans. Evol. Comput. **26**(2), 364–378 (2021)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.